

AUDIT LOG MANAGEMENT SYSTEM

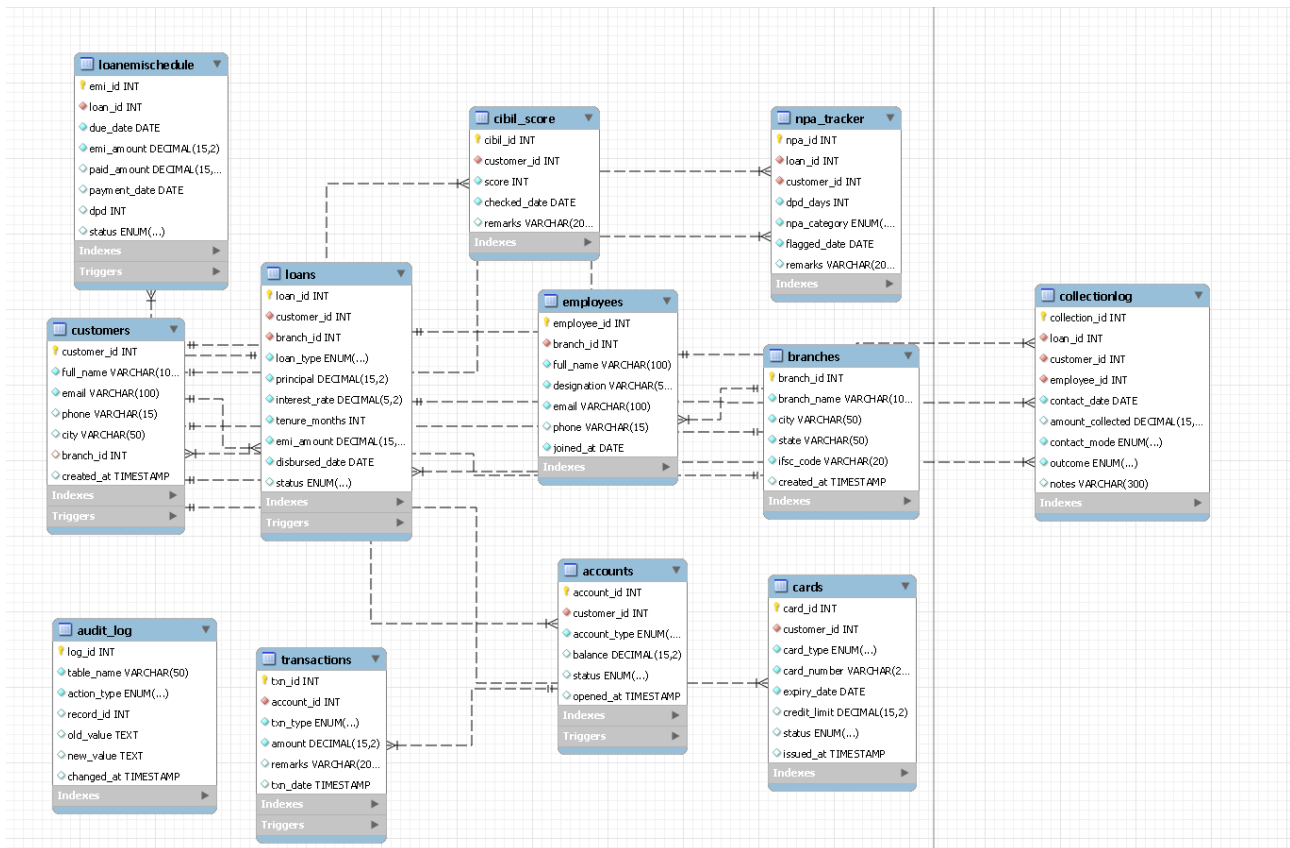
ID: FT0002

Name: Elanchetchenni B

College: EASA College of Engineering and Technology

GitHub URL: <https://github.com/elanchetchenni/BANKING-LOG-System---SQL/tree/main>

Table Schema:



BASIC LEVEL

- 1. Retrieve all customers.
- 2. Retrieve all accounts.
- 3. Retrieve all transactions.
- 4. Retrieve all branches.
- 5. Retrieve all employees.
- 6. Retrieve all loans.
- 7. Retrieve all active loans.
- 8. Retrieve all NPA loans.
- 9. Retrieve all customers in Chennai.
- 10. Retrieve all overdue EMIs.
- 11. Retrieve all blocked or expired cards.
- 12. Retrieve CIBIL scores below 650 (risky customers).
- 13. Retrieve all audit logs for the Customers table.
- 14. Retrieve all deposit transactions.
- 15. Retrieve all collection logs with outcome = 'Paid'.
- 16. Count total customers per city.
- 17. Retrieve EMI schedule records with DPD greater than 30.
- 18. Retrieve all credit cards.
- 19. Retrieve all NPA tracker records flagged in 2024.
- 20. Retrieve total balance per account type.

INTERMEDIATE LEVEL

- 1. Customer name with their account balance and account type.
- 2. Customers with their loan details (type, principal, EMI, status).
- 3. Overdue EMIs with customer name and loan type, ordered by DPD descending.
- 4. Employees with their branch name and city.
- 5. Total loan amount disbursed per branch with loan count.
- 6. Customers with CIBIL score joined with their loan status.
- 7. NPA customers with full collection history.
- 8. Average CIBIL score per city, ordered by average score descending.
- 9. Transactions above Rs.10,000 with account and customer details.
- 10. Loan-wise EMI payment summary (total paid vs outstanding amount).
- 11. Customers with no loans using LEFT JOIN.
- 12. Branch-wise active loan count.
- 13. Total amount collected per collection agent.
- 14. All audit log changes in the last 30 days.
- 15. Cards expiring in 2025 with customer details.

ADVANCED SQL (WINDOW FUNCTIONS, CTEs, STORED PROCEDURES, TRIGGERS)

```
-- 1. Rank customers by CIBIL score using RANK() window function
SELECT c.full_name, cs.score,
RANK() OVER (ORDER BY cs.score DESC) AS cibil_rank
FROM CIBIL_Score cs
JOIN Customers c ON cs.customer_id = c.customer_id;

-- 2. Running total of transactions per account using SUM() OVER
SELECT t.txn_id, c.full_name, t.txn_type, t.amount,
SUM(t.amount) OVER (PARTITION BY t.account_id ORDER BY t.txn_date) AS running_total
FROM Transactions t
JOIN Accounts a ON t.account_id = a.account_id
JOIN Customers c ON a.customer_id = c.customer_id;

-- 3. CTE: Find customers with both NPA loan and low CIBIL score
WITH NPA_Customers AS (
SELECT DISTINCT customer_id FROM Loans WHERE status = 'NPA'
),
Low_CIBIL AS (
SELECT customer_id FROM CIBIL_Score WHERE score < 650
)
SELECT c.full_name, c.city, cs.score
FROM Customers c
JOIN NPA_Customers nc ON c.customer_id = nc.customer_id
JOIN Low_CIBIL lc ON c.customer_id = lc.customer_id
JOIN CIBIL_Score cs ON c.customer_id = cs.customer_id;

-- 4. Find duplicate emails in Customers (data quality check)
SELECT email, COUNT(*) AS count
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1;

-- 5. Top branch by total loan amount using subquery
SELECT branch_name, total_disbursed
FROM (
SELECT b.branch_name, SUM(l.principal) AS total_disbursed
FROM Loans l
JOIN Branches b ON l.branch_id = b.branch_id
GROUP BY b.branch_name
) AS branch_summary
ORDER BY total_disbursed DESC
LIMIT 1;

-- 6. Customers whose EMI DPD is more than 60 days (potential NPA flag)
SELECT DISTINCT c.full_name, c.phone, l.loan_type, e.dpd
FROM LoanEMISchedule e
JOIN Loans l ON e.loan_id = l.loan_id
JOIN Customers c ON l.customer_id = c.customer_id
WHERE e.dpd > 60
ORDER BY e.dpd DESC;

-- 7. Month-wise transaction volume
```

```

SELECT DATE_FORMAT(txn_date, '%Y-%m') AS txn_month,
COUNT(*) AS total_txns,
SUM(amount) AS total_amount
FROM Transactions
GROUP BY txn_month
ORDER BY txn_month;

```

-- 8. Audit log count by action type using CASE WHEN

```

SELECT
SUM(CASE WHEN action_type = 'INSERT' THEN 1 ELSE 0 END) AS inserts,
SUM(CASE WHEN action_type = 'UPDATE' THEN 1 ELSE 0 END) AS updates,
SUM(CASE WHEN action_type = 'DELETE' THEN 1 ELSE 0 END) AS deletes
FROM Audit_Log;

```

-- 9. Employee who handled the most collection contacts

```

SELECT e.full_name, COUNT(cl.collection_id) AS total_contacts
FROM CollectionLog cl
JOIN Employees e ON cl.employee_id = e.employee_id
GROUP BY e.full_name
ORDER BY total_contacts DESC
LIMIT 1;

```

-- 10. Loan portfolio health summary (Active vs NPA vs Closed + NPA ratio %)

```

SELECT
SUM(CASE WHEN status = 'Active' THEN principal ELSE 0 END) AS active_portfolio,
SUM(CASE WHEN status = 'NPA' THEN principal ELSE 0 END) AS npa_portfolio,
SUM(CASE WHEN status = 'Closed' THEN principal ELSE 0 END) AS closed_portfolio,
ROUND(SUM(CASE WHEN status = 'NPA' THEN principal ELSE 0 END) /
SUM(principal) * 100, 2) AS npa_ratio_pct
FROM Loans;

```

-- Audit Log Triggers

-- Trigger 1: Log every new customer INSERT into Audit_Log

```

delimiter //
create trigger trg_customer_insert
after insert on Customers
for each row
begin
insert into Audit_Log (table_name, action_type, record_id, new_value)
values ('Customers', 'INSERT', NEW.customer_id,
concat('Name:', NEW.full_name, ' | Email:', NEW.email,
' | City:', NEW.city));
end //
delimiter ;

```

-- Trigger 2: Log every customer UPDATE (captures old vs new values)

```

delimiter //
create trigger trg_customer_update
after update on Customers
for each row
begin
insert into Audit_Log (table_name, action_type, record_id, old_value, new_value)
values ('Customers', 'UPDATE', NEW.customer_id,
concat('Name:', OLD.full_name, ' | Email:', OLD.email,
' | City:', OLD.city),
concat('Name:', NEW.full_name, ' | Email:', NEW.email,
' | City:', NEW.city));
end //
delimiter ;

```

```

concat('Name:', NEW.full_name, ' | Email:', NEW.email,
' | City:', NEW.city));
end //
delimiter ;

-- Trigger 3: Log customer DELETE before removal
delimiter //
create trigger trg_customer_delete
before delete on Customers
for each row
begin
insert into Audit_Log (table_name, action_type, record_id, old_value)
values ('Customers', 'DELETE', OLD.customer_id,
concat('Name:', OLD.full_name, ' | Email:', OLD.email));
end //
delimiter ;

-- Trigger 4: Log account balance or status changes
delimiter //
create trigger trg_account_update
after update on Accounts
for each row
begin
insert into Audit_Log (table_name, action_type, record_id, old_value, new_value)
values ('Accounts', 'UPDATE', NEW.account_id,
concat('Balance:', OLD.balance, ' | Status:', OLD.status),
concat('Balance:', NEW.balance, ' | Status:', NEW.status));
end //
delimiter ;

-- Trigger 5: Log every new transaction INSERT
delimiter //
create trigger trg_transaction_insert
after insert on Transactions
for each row
begin
insert into Audit_Log (table_name, action_type, record_id, new_value)
values ('Transactions', 'INSERT', NEW.txn_id,
concat('Type:', NEW.txn_type, ' | Amount:', NEW.amount,
' | Remarks:', NEW.remarks));
end //
delimiter ;

-- Trigger 6: Log loan status changes (e.g., Active -> NPA)
delimiter //
create trigger trg_loan_status_update
after update on Loans
for each row
begin
if OLD.status != NEW.status then
insert into Audit_Log (table_name, action_type, record_id, old_value, new_value)
values ('Loans', 'UPDATE', NEW.loan_id,
concat('Status:', OLD.status),
concat('Status:', NEW.status));
end if;
end //

```

```
delimiter ;

-- Trigger 7: Log EMI schedule updates (paid amount, DPD, status)
delimiter //
create trigger trg_emi_payment_update
after update on LoanEMISchedule
for each row
begin
insert into Audit_Log (table_name, action_type, record_id, old_value, new_value)
values ('LoanEMISchedule', 'UPDATE', NEW.emi_id,
concat('Status:', OLD.status, ' | Paid:', OLD.paid_amount,
' | DPD:', OLD.dpd),
concat('Status:', NEW.status, ' | Paid:', NEW.paid_amount,
' | DPD:', NEW.dpd));
end //
delimiter ;
```

-- Stored Procedures

-- Procedure 1: Fetch full audit trail for a specific table

```
delimiter //
create procedure sp_audit_report(IN tbl_name varchar(50))
begin
select log_id, table_name, action_type, record_id,
old_value, new_value, changed_at
from Audit_Log
where table_name = tbl_name
order by changed_at desc;
end //
delimiter ;
```

call sp_audit_report('Customers');

-- Procedure 2: Fetch audit logs between two dates

```
delimiter //
create procedure sp_audit_by_date(IN from_date date, IN to_date date)
begin
select log_id, table_name, action_type, record_id,
old_value, new_value, changed_at
from Audit_Log
where date(changed_at) between from_date and to_date
order by changed_at desc;
end //
delimiter ;
```

call sp_audit_by_date('2024-01-01', '2026-12-31');

-- Procedure 3: Action summary (INSERT / UPDATE / DELETE counts per table)

```
delimiter //
create procedure sp_action_summary()
begin
select table_name,
action_type,
count(*) as total_actions
from Audit_Log
group by table_name, action_type
order by table_name, action_type;
end //
delimiter ;
```

call sp_action_summary();

-- Procedure 4: NPA report with customer and loan details

```
delimiter //
create procedure sp_npa_report()
begin
select c.customer_id, c.full_name, c.phone,
l.loan_id, l.loan_type, l.principal,
n.dpd_days, n.npa_category, n.flagged_date, n.remarks
from NPA_Tracker n
join Loans l on n.loan_id = l.loan_id
join Customers c on n.customer_id = c.customer_id
order by n.dpd_days desc;
```



```

end //
delimiter ;

call sp_npa_report();

-- Procedure 5: Collection efficiency report per field agent
delimiter //
create procedure sp_collection_efficiency()
begin
select e.employee_id, e.full_name, e.designation,
count(cl.collection_id) as total_contacts,
sum(cl.amount_collected) as total_collected,
sum(case when cl.outcome = 'Paid' then 1 else 0 end)
as successful_collections
from CollectionLog cl
join Employees e on cl.employee_id = e.employee_id
group by e.employee_id, e.full_name, e.designation
order by total_collected desc;
end //
delimiter ;

call sp_collection_efficiency();

```